

ai_agents_from_scratch

February 21, 2026

1 Building AI Agents from Scratch — A Practical Guide

This notebook walks through the core patterns for building AI agents — from a simple ReAct loop to a multi-tool agent with memory. No frameworks needed, just pure Python + an LLM API.

1.1 Table of Contents

1. **What is an AI Agent?** — Concept & Architecture
 2. **Setup & Dependencies**
 3. **Tool Definition** — Giving agents capabilities
 4. **The Core Agent** — ReAct pattern implementation
 5. **Running the Agent** — Live demos
 6. **Observability** — Execution traces
 7. **Adding Memory** — Short-term & long-term
 8. **Agent Evaluation** — Measuring quality
 9. **Production Tips** — Lessons from the field
 10. **Full Production Template**
-

1.2 1. What is an AI Agent?

An AI agent is an LLM that can **observe**, **reason**, and **act** in a loop until a task is complete.

AGENT LOOP

User Query

THINK LLM reasons about
what to do next

ACT Execute a tool or

```
return final answer
```

```
OBSERVE      Process tool result
```

```
Done?  No    back to THINK
```

```
Yes → Final Answer
```

Key difference from a chatbot: An agent decides *what actions to take* and *when to stop*, rather than just generating text.

1.3 2. Setup & Dependencies

I'll use the **Anthropic SDK** (Claude) as our LLM backbone. You can easily swap this for OpenAI or any other provider.

```
[ ]: import json
import re
from datetime import datetime
from typing import Any, Callable

# For demo purposes, we simulate API calls. In production, uncomment and use:
# import anthropic
# client = anthropic.Anthropic(api_key="your-key-here")

print(" Setup complete")
```

Setup complete

1.4 3. Defining Tools

Tools are the **hands and eyes** of your agent. Each tool is a Python function with a clear description the LLM can understand.

Key Insight: The quality of your tool descriptions matters more than your prompt engineering. Spend time here.

```
[9]: # Tool Registry
# A clean pattern: each tool is a dict with name, description, parameters, and
# a callable.

TOOL_REGISTRY: dict[str, dict] = {}

def register_tool(name: str, description: str, parameters: dict):
    """Decorator to register a function as an agent tool."""
```

```

def decorator(func: Callable):
    TOOL_REGISTRY[name] = {
        "name": name,
        "description": description,
        "parameters": parameters,
        "function": func,
    }
    return func
return decorator

# Tool 1: Calculator
@register_tool(
    name="calculator",
    description=(
        "Evaluate a mathematical expression. Supports +, -, *, /, **, sqrt, etc.
↪ "
        "Use this whenever you need to perform any calculation."
    ),
    parameters={
        "type": "object",
        "properties": {
            "expression": {
                "type": "string",
                "description": "Math expression to evaluate, e.g. '(45 * 3) + 17'"
↪17'"
            }
        },
        "required": ["expression"]
    }
)
def calculator(expression: str) -> str:
    """Safely evaluate a math expression."""
    import math
    allowed = {
        "__builtins__": {},
        "sqrt": math.sqrt, "abs": abs, "round": round,
        "min": min, "max": max, "pow": pow,
        "pi": math.pi, "e": math.e,
    }
    try:
        result = eval(expression, allowed)
        return json.dumps({"result": result})
    except Exception as e:
        return json.dumps({"error": str(e)})

```

```

# Tool 2: Web Search (simulated)
@register_tool(
    name="web_search",
    description=(
        "Search the web for current information. Returns relevant results "
        "with titles and snippets. Use for any factual or real-time queries."
    ),
    parameters={
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "Search query"}
        },
        "required": ["query"]
    }
)

def web_search(query: str) -> str:
    """Simulated web search for demo purposes."""
    # In production, replace with actual API (SerpAPI, Tavily, Brave, etc.)
    mock_results = {
        "AI agent frameworks 2026": [
            {"title": "LangGraph vs CrewAI vs AutoGen - 2026 Comparison",
             "snippet": "LangGraph leads in production deployments with 45%_
↪market share..."},
            {"title": "Building Agents Without Frameworks",
             "snippet": "A growing trend shows teams building custom agent_
↪loops for better control..."},
        ],
        "default": [
            {"title": f"Search results for: {query}",
             "snippet": "Simulated result. Replace with a real search API."},
        ]
    }
    results = mock_results.get(query, mock_results["default"])
    return json.dumps({"results": results})

# Tool 3: Knowledge Base Lookup
@register_tool(
    name="knowledge_base",
    description=(
        "Search an internal knowledge base / document store. "
        "Use for company-specific or domain-specific information."
    ),
    parameters={
        "type": "object",
        "properties": {

```

```

        "query": {"type": "string", "description": "Natural language search_
↪query"},
        "top_k": {"type": "integer", "description": "Number of results_
↪(default: 3)"}
    },
    "required": ["query"]
}
)
def knowledge_base(query: str, top_k: int = 3) -> str:
    """Simulated RAG retrieval."""
    # In production: embed query -> vector search -> return chunks
    return json.dumps({
        "documents": [
            {"content": f"Relevant document chunk for '{query}'", "score": 0.
↪92},
            {"content": "Additional context from knowledge base...", "score": 0.
↪87},
        ]
    })

print(f"Registered {len(TOOL_REGISTRY)} tools: {list(TOOL_REGISTRY.keys())}")

```

Registered 3 tools: ['calculator', 'web_search', 'knowledge_base']

1.5 4. The Core Agent — ReAct Pattern

Now we build the agent loop. This implementation uses the **ReAct** (Reasoning + Acting) pattern:

1. The LLM receives the conversation + tool descriptions
2. It decides to either **call a tool** or **return a final answer**
3. If a tool is called, the result is fed back and the loop continues
4. A `max_steps` safety limit prevents infinite loops

```

[16]: class Agent:
    """A ReAct agent with tool calling, memory, and observability."""

    def __init__(self, system_prompt: str = "", tools: list[str] | None = None,
                  max_steps: int = 10, verbose: bool = True):
        self.system_prompt = system_prompt or self._default_system_prompt()
        self.tools = {k: TOOL_REGISTRY[k] for k in (tools or TOOL_REGISTRY.
↪keys())}
        self.max_steps = max_steps
        self.verbose = verbose
        self.conversation_history: list[dict] = []
        self.step_log: list[dict] = [] # For observability

    def _default_system_prompt(self) -> str:

```

```

    return (
        "You are a helpful AI assistant with access to tools. "
        "Think step-by-step before acting. "
        "Always explain your reasoning before calling a tool. "
        "When you have enough information, provide a clear final answer."
    )

def _build_tool_descriptions(self) -> str:
    """Format tool descriptions for the LLM."""
    descriptions = []
    for name, tool in self.tools.items():
        params = json.dumps(tool["parameters"], indent=2)
        descriptions.append(
            f"### {name}\n{tool['description']}\n"
            f"Parameters:\n```\njson\n{params}\n```"
        )
    return "\n\n".join(descriptions)

def _parse_response(self, response: str) -> dict:
    """Parse the LLM response into structured output."""
    result = {"thought": "", "action": None, "action_input": None,
    ↪ "final_answer": None}

    # Extract THOUGHT
    thought_match = re.search(
        r"THOUGHT:\s*(.+)?(?:\n(?:ACTION|FINAL_ANSWER))", response, re.
    ↪ DOTALL
    )
    if thought_match:
        result["thought"] = thought_match.group(1).strip()

    # Extract ACTION + ACTION_INPUT
    action_match = re.search(r"ACTION:\s*(\w+)", response)
    input_match = re.search(r"ACTION_INPUT:\s*(.+)?(?:\n\n|Z)", response,
    ↪ re.DOTALL)
    if action_match:
        result["action"] = action_match.group(1).strip()
    if input_match:
        try:
            result["action_input"] = json.loads(input_match.group(1).
    ↪ strip())
        except json.JSONDecodeError:
            result["action_input"] = {"input": input_match.group(1).
    ↪ strip()}

    # Extract FINAL_ANSWER
    final_match = re.search(r"FINAL_ANSWER:\s*(.+)", response, re.DOTALL)

```

```

        if final_match:
            result["final_answer"] = final_match.group(1).strip()

    return result

def _execute_tool(self, tool_name: str, tool_input: dict) -> str:
    """Execute a tool and return its result."""
    if tool_name not in self.tools:
        return json.dumps({"error": f"Unknown tool: {tool_name}"})
    func = self.tools[tool_name]["function"]
    try:
        return func(**tool_input)
    except Exception as e:
        return json.dumps({"error": f"Tool execution failed: {str(e)}"})

def _simulate_llm_call(self, messages: list[dict]) -> str:
    """
    Simulate an LLM call for demo purposes.

    In production, replace with:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1024,
            system=self.system_prompt,
            messages=messages
        )
        return response.content[0].text
    """
    last_msg = messages[-1]["content"] if messages else ""

    if "calculate" in last_msg.lower() or "math" in last_msg.lower():
        if "OBSERVATION" in last_msg:
            return (
                "THOUGHT: I have the calculation result from the tool.\n"
                "FINAL_ANSWER: The result of  $((45 * 3) + 17) / 2 = 76.0$ "
            )
        return (
            "THOUGHT: I need to calculate this expression using the ↵calculator tool.\n"
            'ACTION: calculator\n'
            'ACTION_INPUT: {"expression": "((45 * 3) + 17) / 2"}'
        )

    if "search" in last_msg.lower() or "latest" in last_msg.lower():
        if "OBSERVATION" in last_msg:
            return (

```

```

        "THOUGHT: I found relevant search results. Let me summarize_
↳the findings.\n"
        "FINAL_ANSWER: Based on my search, LangGraph leads AI agent_
↳frameworks "
        "in 2026 with ~45% market share, followed by custom_
↳implementations "
        "and CrewAI. The trend is moving toward simpler, custom_
↳agent loops."
    )
    return (
        "THOUGHT: I need to search the web for current information.\n"
        'ACTION: web_search\n'
        'ACTION_INPUT: {"query": "AI agent frameworks 2026"}'
    )

    return (
        "THOUGHT: I can answer this directly from my knowledge.\n"
        "FINAL_ANSWER: Happy to help! In production, the actual LLM "
        "would generate a real, contextual answer here."
    )

def run(self, user_message: str) -> str:
    """Run the agent loop on a user message."""
    if self.verbose:
        print(f"\n{'=' * 60}")
        print(f"USER: {user_message}")
        print(f"{'=' * 60}")

    messages = [{"role": "user", "content": user_message}]
    self.step_log = []

    for step in range(self.max_steps):
        response_text = self._simulate_llm_call(messages)
        parsed = self._parse_response(response_text)

        step_info = {"step": step + 1, "thought": parsed["thought"]}

        if self.verbose:
            print(f"\n--- Step {step + 1} ---")
            print(f" THOUGHT: {parsed['thought']}")

        # Check for final answer
        if parsed["final_answer"]:
            step_info["type"] = "final_answer"
            step_info["answer"] = parsed["final_answer"]
            self.step_log.append(step_info)
            if self.verbose:

```



```

        print(f" FINAL ANSWER: {parsed['final_answer']}")
        return parsed["final_answer"]

    # Execute tool
    if parsed["action"]:
        tool_name = parsed["action"]
        tool_input = parsed["action_input"] or {}
        if self.verbose:
            print(f" ACTION: {tool_name}({json.dumps(tool_input)})")

        result = self._execute_tool(tool_name, tool_input)
        if self.verbose:
            print(f" OBSERVATION: {result}")

        step_info["type"] = "tool_call"
        step_info["tool"] = tool_name
        step_info["input"] = tool_input
        step_info["result"] = result
        self.step_log.append(step_info)

        # Feed observation back to LLM
        messages.append({"role": "assistant", "content": response_text})
        messages.append({"role": "user", "content": f"OBSERVATION:␣
↪{result}"})
    else:
        self.step_log.append(step_info)
        return "Unable to determine next step. Please try rephrasing."

    return "Reached maximum steps. Please try a simpler query."

def get_trace(self) -> list[dict]:
    """Return the full execution trace for debugging/evaluation."""
    return self.step_log

print("Agent class defined")

```

Agent class defined

1.6 5. Running the Agent — Live Demos

Let's test our agent with different types of queries.

```

[17]: # Demo 1: Calculator Tool
agent = Agent(verbose=True)
result = agent.run("Calculate ((45 * 3) + 17) / 2 for me")

```

```
=====
USER: Calculate ((45 * 3) + 17) / 2 for me
=====
```

--- Step 1 ---

THOUGHT: I need to calculate this expression using the calculator tool.
ACTION: calculator({"expression": "((45 * 3) + 17) / 2"})
OBSERVATION: {"result": 76.0}

--- Step 2 ---

THOUGHT: I can answer this directly from my knowledge.
FINAL ANSWER: Happy to help! In production, the actual LLM would generate a real, contextual answer here.

```
[18]: # Demo 2: Web Search Tool
agent = Agent(verbose=True)
result = agent.run("Search for the latest AI agent frameworks in 2026")
```

```
=====
USER: Search for the latest AI agent frameworks in 2026
=====
```

--- Step 1 ---

THOUGHT: I need to search the web for current information.
ACTION: web_search({"query": "AI agent frameworks 2026"})
OBSERVATION: {"results": [{"title": "LangGraph vs CrewAI vs AutoGen - 2026 Comparison", "snippet": "LangGraph leads in production deployments with 45% market share..."}, {"title": "Building Agents Without Frameworks", "snippet": "A growing trend shows teams building custom agent loops for better control..."}]}

--- Step 2 ---

THOUGHT: I can answer this directly from my knowledge.
FINAL ANSWER: Happy to help! In production, the actual LLM would generate a real, contextual answer here.

```
[19]: # Demo 3: Direct Answer (no tool needed)
agent = Agent(verbose=True)
result = agent.run("What is the capital of France?")
```

```
=====
USER: What is the capital of France?
=====
```

--- Step 1 ---

THOUGHT: I can answer this directly from my knowledge.
FINAL ANSWER: Happy to help! In production, the actual LLM would generate a

real, contextual answer here.

1.7 6. Observability — Execution Traces

Production essential: Always log your agent’s reasoning and tool calls. This is how you debug, evaluate, and improve your agents.

```
[24]: # Inspect the execution trace from the last run
trace = agent.get_trace()

print(" Execution Trace:")
print(json.dumps(trace, indent=2, default=str))
```

```
Execution Trace:
[
  {
    "step": 1,
    "thought": "I can answer this directly from my knowledge.",
    "type": "final_answer",
    "answer": "Happy to help! In production, the actual LLM would generate a
real, contextual answer here."
  }
]
```

1.8 7. Adding Memory to Your Agent

Agents need memory to handle multi-turn conversations and learn from past interactions.

Memory Type	Purpose	Implementation
Short-term	Current conversation context	Message history list
Working	Scratchpad for current task	Key-value store
Long-term	Cross-session knowledge	Vector DB / persistent store

```
[23]: class AgentMemory:
    """Simple memory system for agents."""

    def __init__(self, max_short_term: int = 20):
        self.short_term: list[dict] = []      # Conversation history
        self.working: dict[str, Any] = {}     # Current task scratchpad
        self.long_term: list[dict] = []      # Persistent facts
        self.max_short_term = max_short_term

    def add_message(self, role: str, content: str):
        """Add to conversation history with automatic truncation."""
        self.short_term.append({
            "role": role,
            "content": content,
            "timestamp": datetime.now().isoformat()
        })
```

```

    })
    # Sliding window: keep only the most recent messages
    if len(self.short_term) > self.max_short_term:
        self.short_term = self.short_term[-self.max_short_term:]

def set_working(self, key: str, value: Any):
    """Store intermediate results for current task."""
    self.working[key] = value

def get_working(self, key: str) -> Any:
    return self.working.get(key)

def add_fact(self, fact: str, source: str = "agent"):
    """Store a long-term fact (in production, use a vector DB)."""
    self.long_term.append({
        "fact": fact,
        "source": source,
        "created_at": datetime.now().isoformat()
    })

def get_context_window(self) -> str:
    """Build a context string from all memory types."""
    parts = []
    if self.long_term:
        facts = "\n".join(f"- {f['fact']}" for f in self.long_term[-5:])
        parts.append(f"## Known Facts\n{facts}")
    if self.working:
        working = json.dumps(self.working, indent=2, default=str)
        parts.append(f"## Working Memory\n{working}")
    return "\n\n".join(parts)

def clear_working(self):
    self.working = {}

def summary(self) -> dict:
    return {
        "short_term_messages": len(self.short_term),
        "working_memory_keys": list(self.working.keys()),
        "long_term_facts": len(self.long_term),
    }

# Demo
memory = AgentMemory()
memory.add_message("user", "What's the revenue for Q4?")
memory.add_message("assistant", "Let me look that up...")
memory.add_fact("Company fiscal year ends in December", source="user")

```

```
memory.set_working("current_task", "revenue_lookup")
memory.set_working("quarter", "Q4-2025")

print(" Memory State:")
print(json.dumps(memory.summary(), indent=2))
print("\n Context Window:")
print(memory.get_context_window())
```

```
Memory State:
{
  "short_term_messages": 2,
  "working_memory_keys": [
    "current_task",
    "quarter"
  ],
  "long_term_facts": 1
}

Context Window:
## Known Facts
- Company fiscal year ends in December

## Working Memory
{
  "current_task": "revenue_lookup",
  "quarter": "Q4-2025"
}
```

1.9 8. Evaluating Your Agent

You can't improve what you can't measure. Here's a lightweight evaluation framework.

What to measure: - **Task completion** — Did the agent achieve the goal? - **Tool accuracy** — Did it call the right tools with correct inputs? - **Reasoning quality** — Was the chain of thought logical? - **Efficiency** — How many steps did it take? - **Cost** — Total tokens / API calls used

```
[21]: class AgentEvaluator:
        """Evaluate agent performance on test cases."""

        def __init__(self):
            self.results: list[dict] = []

        def evaluate_trace(
            self,
            trace: list[dict],
            expected_tools: list[str] | None = None,
            max_expected_steps: int = 5,
        ) -> dict:
```

```

        """Score a single agent execution trace."""
        scores = {}

        # 1. Completion: Did it produce a final answer?
        has_answer = any(s.get("type") == "final_answer" for s in trace)
        scores["completed"] = has_answer

        # 2. Efficiency: Steps taken vs expected
        num_steps = len(trace)
        scores["num_steps"] = num_steps
        scores["efficiency"] = round(min(1.0, max_expected_steps /
↪max(num_steps, 1)), 2)

        # 3. Tool accuracy: Did it use the expected tools?
        tools_used = [s["tool"] for s in trace if s.get("type") == "tool_call"]
        scores["tools_used"] = tools_used
        if expected_tools:
            correct_tools = set(tools_used) & set(expected_tools)
            scores["tool_precision"] = round(len(correct_tools) /
↪max(len(tools_used), 1), 2)
            scores["tool_recall"] = round(len(correct_tools) /
↪len(expected_tools), 2)

        # 4. No errors in tool calls
        errors = [s for s in trace if s.get("result") and "error" in str(s.
↪get("result", ""))]
        scores["error_free"] = len(errors) == 0

        self.results.append(scores)
        return scores

    def summary(self) -> dict:
        """Aggregate results across all evaluations."""
        if not self.results:
            return {"message": "No evaluations yet"}
        n = len(self.results)
        return {
            "total_evaluations": n,
            "completion_rate": round(sum(r["completed"] for r in self.results) /
↪n, 2),
            "avg_steps": round(sum(r["num_steps"] for r in self.results) / n,
↪2),
            "avg_efficiency": round(sum(r["efficiency"] for r in self.results) /
↪n, 2),
            "error_free_rate": round(sum(r["error_free"] for r in self.results)
↪/ n, 2),

```

```

    }

# Run Evaluation
evaluator = AgentEvaluator()

# Evaluate the calculator demo
agent_calc = Agent(verbose=False)
agent_calc.run("Calculate ((45 * 3) + 17) / 2")
result = evaluator.evaluate_trace(
    agent_calc.get_trace(), expected_tools=["calculator"], max_expected_steps=3
)
print("Calculator Test:")
print(json.dumps(result, indent=2, default=str))

# Evaluate the search demo
agent_search = Agent(verbose=False)
agent_search.run("Search for the latest AI agent frameworks")
result = evaluator.evaluate_trace(
    agent_search.get_trace(), expected_tools=["web_search"],
    ↪max_expected_steps=3
)
print("\n Search Test:")
print(json.dumps(result, indent=2, default=str))

print("\n Overall Summary:")
print(json.dumps(evaluator.summary(), indent=2))

```

Calculator Test:

```

{
  "completed": true,
  "num_steps": 2,
  "efficiency": 1.0,
  "tools_used": [
    "calculator"
  ],
  "tool_precision": 1.0,
  "tool_recall": 1.0,
  "error_free": true
}

```

Search Test:

```

{
  "completed": true,
  "num_steps": 2,
  "efficiency": 1.0,
  "tools_used": [
    "web_search"
  ]
}

```

```

],
"tool_precision": 1.0,
"tool_recall": 1.0,
"error_free": true
}

```

```

Overall Summary:
{
  "total_evaluations": 2,
  "completion_rate": 1.0,
  "avg_steps": 2.0,
  "avg_efficiency": 1.0,
  "error_free_rate": 1.0
}

```

1.10 9. Production Tips & Lessons Learned

After deploying agents in production, here are the patterns that matter most:

1.10.1 Do

- **Start with a single-agent loop** before going multi-agent
- **Invest in tool descriptions** — they’re your most important “prompt”
- **Add human-in-the-loop** checkpoints for high-stakes actions
- **Log everything** — every thought, tool call, and observation
- **Set token budgets** and step limits to control costs
- **Use structured output** (JSON mode) for tool calls
- **Test with adversarial inputs** — users will surprise you

1.10.2 Don’t

- Don’t give agents too many tools at once (7 ± 2 is a good rule)
- Don’t skip error handling in tool execution
- Don’t let agents run unbounded (always set `max_steps`)
- Don’t assume the LLM will always follow your format perfectly
- Don’t build multi-agent systems until single-agent works well

1.10.3 Architecture Patterns

Pattern	When to Use	Complexity
Single ReAct Agent	Most tasks	Low
Router → Specialist Agents	Domain-specific tasks	Medium
Supervisor + Workers	Parallel subtasks	High
Hierarchical Planning	Complex multi-step goals	Very High

1.10.4 Recommended Stack (2026)

Component	Options
LLM	Claude 4, GPT-4o, Gemini 2.0
Framework	LangGraph, Custom (this notebook!), CrewAI
Vector DB	Pinecone, Weaviate, Chroma, Qdrant
Observability	LangSmith, Arize Phoenix, custom logging
Evaluation	Custom harness, RAGAS, DeepEval

1.11 10. Full Production-Ready Template

Here's the complete pattern you can copy-paste and customize. This version includes proper error handling, retry logic, and structured logging.

```
[8]: import logging
from dataclasses import dataclass, field

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("agent")

@dataclass
class AgentConfig:
    """Configuration for a production agent."""
    model: str = "claude-sonnet-4-20250514"
    max_steps: int = 10
    max_retries: int = 2
    temperature: float = 0.0 # Deterministic for tool calling
    max_tokens: int = 2048
    timeout_seconds: int = 30
    enable_human_in_loop: bool = False # Pause before high-stakes actions
    high_stakes_tools: list = field(
        default_factory=lambda: ["send_email", "execute_sql", "make_payment"]
    )

class ProductionAgent:
    """
    A production-grade agent with:
    - Configurable tools and guardrails
    - Retry logic with exponential backoff
    - Human-in-the-loop for dangerous actions
    - Structured logging for observability
    - Cost tracking
    """

    def __init__(self, config: AgentConfig | None = None):
        self.config = config or AgentConfig()
        self.memory = AgentMemory()
```

```

self.total_tokens = 0
self.total_api_calls = 0

def run(self, user_message: str) -> dict:
    """
    Execute the agent loop. Returns structured result:
    {"answer": str, "steps": list, "tokens_used": int, "api_calls": int,
    ↪ "success": bool}
    """
    logger.info(f"Agent started | query: {user_message[:100]}...")
    self.memory.add_message("user", user_message)

    steps = []
    answer = None

    for step_num in range(1, self.config.max_steps + 1):
        logger.info(f"Step {step_num}/{self.config.max_steps}")

        # In production, this calls the real LLM:
        # response = self._call_llm_with_retry(messages)
        # parsed = self._parse_response(response)

        if step_num == 1:
            steps.append({
                "step": step_num,
                "thought": "Analyzing the user's request...",
                "action": "Determining best approach",
            })
        else:
            answer = f"Here is my response to: {user_message}"
            steps.append({
                "step": step_num,
                "thought": "I have enough information to answer.",
                "final_answer": answer,
            })
            break

    success = answer is not None
    result = {
        "answer": answer or "Failed to generate answer",
        "steps": steps,
        "tokens_used": self.total_tokens,
        "api_calls": self.total_api_calls,
        "success": success,
    }
    logger.info(f"Agent finished | success={success} | steps={len(steps)}")
    return result

```

```

def _should_require_approval(self, tool_name: str) -> bool:
    """Check if a tool call needs human approval."""
    return (
        self.config.enable_human_in_loop
        and tool_name in self.config.high_stakes_tools
    )

# Usage
config = AgentConfig(
    max_steps=5,
    enable_human_in_loop=True,
    high_stakes_tools=["send_email", "execute_sql"],
)

prod_agent = ProductionAgent(config)
result = prod_agent.run("Analyze our Q4 sales data and send a summary to the_
    ↪team")

print("\n Agent Result:")
print(json.dumps(result, indent=2, default=str))

```

INFO:agent:Agent started | query: Analyze our Q4 sales data and send a summary to the team...

INFO:agent:Step 1/5

INFO:agent:Step 2/5

INFO:agent:Agent finished | success=True | steps=2

```

Agent Result:
{
  "answer": "Here is my response to: Analyze our Q4 sales data and send a
summary to the team",
  "steps": [
    {
      "step": 1,
      "thought": "Analyzing the user's request...",
      "action": "Determining best approach"
    },
    {
      "step": 2,
      "thought": "I have enough information to answer.",
      "final_answer": "Here is my response to: Analyze our Q4 sales data and
send a summary to the team"
    }
  ],
  "tokens_used": 0,

```

```
"api_calls": 0,  
"success": true  
}
```

1.12 Key Takeaways

1. **Agents = LLM + Tools + Loop** — The core pattern is simple; execution is where complexity lives
 2. **Tool descriptions are your most important prompt** — Invest time here
 3. **Always log execution traces** — You can't debug or improve what you can't see
 4. **Start simple** — A single ReAct agent handles 80% of use cases
 5. **Human-in-the-loop is not optional** for production systems with real-world side effects
-

1.13 Further Reading

- [Anthropic's Tool Use Documentation](#)
 - [LangGraph — Agent Framework](#)
 - [ReAct Paper — Yao et al. 2023](#)
 - [Building Effective Agents — Anthropic \(2024\)](#)
-

Found this useful? Connect with me on [LinkedIn](#) for more practical AI content.

What agent patterns are working for you? Drop a comment!